

ARES Protocol

Connection and operating guide for REST/JSON, XML, Zenoh, and ROS2DDS

[REST/JSON](#)
[XML](#)
[Zenoh](#)
[ROS2DDS](#)

This document describes how clients connect to an already-running ARES server. It documents the external protocol only: URLs, keys, payloads, and expected responses.

Contents

1	Connection points	2
2	Shared data rules	2
2.1	Resource identifiers	2
2.2	Common enum values	2
2.3	Error response	2
3	Symmetric ARES call catalog	2
4	Operating story	3
4.1	Story overview	3
4.2	Full REST story: robot claims dock A	4
4.3	Contention story: second robot is denied until release	4
4.4	Shared-zone story: capacity	5
4.5	Full XML story: same claim flow	5
5	REST / JSON protocol	6
5.1	Health	6
5.2	Map lookup	6
5.3	Route planning	7
5.4	Robots	7
5.5	Claims	8
5.6	Leases and snapshot	8
6	XML protocol	9
6.1	XML endpoint table	9
6.2	XML calls	9
7	Zenoh and ROS2DDS	11
7.1	ROS2 service catalog	11
7.2	ares_interfaces/srv/Json	13
7.3	Full ROS2 JSON services	13
7.4	ROS2 connection story	15
7.5	ROS2 troubleshooting	15
7.6	Internal ROS2DDS response encoding	15
8	ARES native Zenoh services	16
8.1	Complete Zenoh service catalog	16
8.2	Native Zenoh operating sequence	20

1 Connection points

The operator running ARES should provide these values.

Name	Example	Used for
REST_BASE	<code>http://84.22.103.80:8080/ares/v1</code>	REST/JSON calls.
XML_BASE	<code>http://84.22.103.80:8080/ares/v1</code>	XML calls, if the XML adapter is exposed.
ZENO_ENDPOINT	<code>tcp/84.22.103.80:7447</code>	Zenoh bridge or peer connection.
ROS2_SERVICE	<code>/ares/v1/health</code>	ARES JSON service through <code>zenoh-bridge-ros2dds</code> .

Zenoh endpoint syntax: use `tcp/84.22.103.80:7447`. Do not use `tcp://84.22.103.80:7447`.

2 Shared data rules

2.1 Resource identifiers

Zones, nodes, and edges are UUID-based internally, but client-facing calls can use either a UUID or a numeric alias. Numeric aliases should be sent as strings to keep JSON and XML compatible.

```
"100"
```

```
"00000000-0000-0000-0000-000000000100"
```

2.2 Common enum values

Field	Allowed values
Claim target kind	<code>Zone</code> , <code>Node</code> , <code>Edge</code>
Claim access mode	<code>Shared</code> , <code>Exclusive</code>
Robot progress	<code>Idle</code> , <code>FollowingRoute</code> , <code>Waiting</code> , <code>Queued</code> , <code>Blocked</code> , <code>Replanning</code>
Lease disposition	<code>Active</code> , <code>Released</code> , <code>Expired</code> , <code>Revoked</code>

2.3 Error response

REST and XML API errors return HTTP `400`. JSON error body:

```
{"message":"what went wrong"}
```

3 Symmetric ARES call catalog

The four protocol surfaces are symmetric. HTTP transports use paths. Zenoh and ROS2 use service names/keys. For `GET /.../{id}` style HTTP calls, Zenoh and ROS2 use a tiny JSON request object such as `{"id":"100"}`, `{"robot_id":1}`, or `{"claim_id":10}`.

Operation	REST JSON	XML	Zenoh key	ROS2 service
Health	<code>GET /health</code>	<code>GET /health</code>	<code>ares/v1/health</code>	<code>/ares/v1/health</code>
Snapshot	<code>GET /fleet/snapshot</code>	<code>GET /fleet/snapshot</code>	<code>ares/v1/fleet/snapshot</code>	<code>/ares/v1/fleet/snapshot</code>
List zones	<code>GET /zones</code>	<code>GET /zones</code>	<code>ares/v1/zones/list</code>	<code>/ares/v1/zones/list</code>
Get zone	<code>GET /zones/{id}</code>	<code>GET /zones/{id}</code>	<code>ares/v1/zones/get</code>	<code>/ares/v1/zones/get</code>

Operation	REST JSON	XML	Zenoh key	ROS2 service
List nodes	GET /nodes	GET /nodes	ares/v1/nodes/list	/ares/v1/nodes/list
Get node	GET /nodes/{id}	GET /nodes/{id}	ares/v1/nodes/get	/ares/v1/nodes/get
List edges	GET /edges	GET /edges	ares/v1/edges/list	/ares/v1/edges/list
Get edge	GET /edges/{id}	GET /edges/{id}	ares/v1/edges/get	/ares/v1/edges/get
Plan route	POST /routes/plan	POST /routes/plan	ares/v1/routes/plan	/ares/v1/routes/plan
Register robot	POST /robots	POST /robots	ares/v1/robots/register	/ares/v1/robots/register
List robots	GET /robots	GET /robots	ares/v1/robots/list	/ares/v1/robots/list
Get robot	GET /robots/{id}	GET /robots/{id}	ares/v1/robots/get	/ares/v1/robots/get
Unregister robot	DELETE /robots/{id}	DELETE /robots/{id}	ares/v1/robots/unregister	/ares/v1/robots/unregister
Heartbeat	POST /robots/{id}/heartbeat	POST /robots/{id}/heartbeat	ares/v1/robots/heartbeat	/ares/v1/robots/heartbeat
Assign route	POST /robots/{id}/route	POST /robots/{id}/route	ares/v1/robots/assign_route	/ares/v1/robots/assign_route
Schedule	POST /robots/{id}/schedule	POST /robots/{id}/schedule	ares/v1/robots/schedule	/ares/v1/robots/schedule
List claims	GET /claims	GET /claims	ares/v1/claims/list	/ares/v1/claims/list
Get claim	GET /claims/{id}	GET /claims/{id}	ares/v1/claims/get	/ares/v1/claims/get
Remove claim	DELETE /claims/{id}	DELETE /claims/{id}	ares/v1/claims/remove	/ares/v1/claims/remove
Evaluate claim	POST /claims/evaluate	POST /claims/evaluate	ares/v1/claims/evaluate	/ares/v1/claims/evaluate
Submit claim	POST /claims	POST /claims	ares/v1/claims/request	/ares/v1/claims/request
List leases	GET /leases	GET /leases	ares/v1/leases/list	/ares/v1/leases/list
Add lease	POST /leases	POST /leases	ares/v1/leases/add	/ares/v1/leases/add
Release lease	POST /leases/release or DELETE /leases/{id}	POST /leases/release or DELETE /leases/{id}	ares/v1/leases/release	/ares/v1/leases/release

4 Operating story

This is the normal ARES flow: check the server, read the map, register the robot, plan movement, ask for access to a zone, then keep the server updated as the robot moves.

4.1 Story overview

Step	Action	Why
1	Health check	Confirm that the ARES server is reachable.
2	Read map resources	Find zone, node, and edge IDs. XML does not expose map lookup yet, so use REST for this step.
3	Register robot	Tell ARES the robot exists and where it starts.
4	Plan route	Ask ARES for a graph route between nodes.
5	Evaluate claim	Dry-run zone/node/edge access before committing.
6	Submit claim	Store the request if ARES grants it.
7	Move / heartbeat	Update robot progress while it moves.
8	Release / cleanup	Remove claims or release leases when finished.
9	Snapshot	Read robots, active claims, and leases in one response.

4.2 Full REST story: robot claims dock A

Set the base URL:

```
REST_BASE=http://84.22.103.80:8080/ares/v1
```

1. Check health:

```
curl $REST_BASE/health
```

2. Read zones and nodes. In the fixed example, zone `100` is `dock_a`, node `1001` is the dock entry, node `1003` is the other side of the map.

```
curl $REST_BASE/zones
```

```
curl $REST_BASE/nodes
```

3. Register robot `1` at node `1001`.

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d '{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","up
```

4. Plan a route from node `1001` to node `1003`.

```
curl -X POST $REST_BASE/routes/plan -H 'content-type: application/json' -d '{"start_node_id":"1001","goal_node_id":"1003","use_penalties":false}'
```

5. Dry-run an exclusive claim on zone `100`.

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d '{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

If the response contains `"decision":"Grant"`, submit the claim.

6. Submit/store the granted claim.

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d '{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

7. Confirm the claim is active.

```
curl $REST_BASE/claims
```

8. Update robot progress as it moves.

```
curl -X POST $REST_BASE/robots/1/heartbeat -H 'content-type: application/json' -d '{"current_node_id":"1002","current_edge_id":null,"updated_at_tick":70}'
```

9. Read the whole fleet state.

```
curl $REST_BASE/fleet/snapshot
```

10. When done, remove the active claim.

```
curl -X DELETE $REST_BASE/claims/10
```

4.3 Contention story: second robot is denied until release

Robot `2` tries to claim the same exclusive zone while robot `1` still owns claim `10`.

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d
'{"robot_id":2,"mission_id":9002,"current_node_id":"00000000-0000-0000-0000-000000001003","progress_state":"Idle","up
```

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d
'{"id":11,"robot_id":2,"mission_id":9002,"access_mode":"Exclusive","priority":5,"requested_at_tick":11,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

Expected decision while claim 10 is active:

```
{"decision":"Deny", "...":"..."}
```

Release robot 1 claim:

```
curl -X DELETE $REST_BASE/claims/10
```

Now robot 2 can submit:

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":11,"robot_id":2,"mission_id":9002,"access_mode":"Exclusive","priority":5,"requested_at_tick":12,"window":
{"start_tick":12,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}'
```

4.4 Shared-zone story: capacity

For shared zones, more than one robot may be granted if the zone policy allows capacity. In the fixed example, zone 101 is shared.

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":20,"robot_id":1,"mission_id":9101,"access_mode":"Shared","priority":1,"requested_at_tick":20,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":21,"robot_id":2,"mission_id":9102,"access_mode":"Shared","priority":1,"requested_at_tick":21,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

If capacity is full, another overlapping claim is denied:

```
curl -X POST $REST_BASE/claims/evaluate -H 'content-type: application/json' -d
'{"id":22,"robot_id":3,"mission_id":9103,"access_mode":"Shared","priority":1,"requested_at_tick":22,"window":
{"start_tick":20,"end_tick":120},"targets":[{"kind":"Zone","resource_id":"101"}]}'
```

4.5 Full XML story: same claim flow

Set:

```
XML_BASE=http://84.22.103.80:8080/ares/v1
```

XML has the claim/robot/route/lease calls. For map lookup, use REST first because XML map lookup is not exposed yet.

1. Check health:

```
curl -H 'accept: application/xml' $XML_BASE/health
```

2. Register robot:

```
curl -X POST $XML_BASE/robots -H 'content-type: application/xml' -d '<RobotState><robot_id>1</
robot_id><mission_id>9001</mission_id><current_node_id>00000000-0000-0000-0000-000000001001</
current_node_id><progress_state>Idle</progress_state><updated_at_tick>1</updated_at_tick></RobotState>'
```

3. Plan route:

```
curl -X POST $XML_BASE/routes/plan -H 'content-type: application/xml' -d
'<PlanRouteRequest><start_node_id>1001</start_node_id><goal_node_id>1003</
goal_node_id><use_penalties>>false</use_penalties></PlanRouteRequest>'
```

4. Dry-run claim:

```
curl -X POST $XML_BASE/claims/evaluate -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

5. Submit claim:

```
curl -X POST $XML_BASE/claims -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

6. Heartbeat:

```
curl -X POST $XML_BASE/robots/1/heartbeat -H 'content-type: application/xml' -d '<HeartbeatRequest><current_node_id>1002</current_node_id><updated_at_tick>70</updated_at_tick></HeartbeatRequest>'
```

7. Snapshot:

```
curl -H 'accept: application/xml' $XML_BASE/fleet/snapshot
```

8. Add a lease directly if the integration uses leases:

```
curl -X POST $XML_BASE/leases -H 'content-type: application/xml' -d '<Lease><id>501</id><claim_id>10</claim_id><robot_id>1</robot_id><access_mode>Exclusive</access_mode><targets><kind>Zone</kind><resource_id>00000000-0000-0000-0000-00000000100</resource_id></targets><granted_at_tick>12</granted_at_tick><expires_at_tick>200</expires_at_tick><disposition>Active</disposition><active>true</active></Lease>'
```

9. Release lease:

```
curl -X POST $XML_BASE/leases/release -H 'content-type: application/xml' -d '<ReleaseLeaseRequest><lease_id>501</lease_id><released_at_tick>50</released_at_tick></ReleaseLeaseRequest>'
```

5 REST / JSON protocol

Set the base URL:

```
REST_BASE=http://84.22.103.80:8080/ares/v1
```

All REST request bodies are JSON. All successful responses are JSON.

5.1 Health

Method	Path / Key	Request	Response
GET	/health	none	Health

```
curl $REST_BASE/health
```

```
{"status": "ok", "version": "0.0.2"}
```

5.2 Map lookup

Method	Path	Meaning
GET	/zones	List zones.
GET	/zones/{id}	Get zone by UUID or numeric alias.

Method	Path	Meaning
GET	/nodes	List graph nodes.
GET	/nodes/{id}	Get node by UUID or numeric alias.
GET	/edges	List graph edges.
GET	/edges/{id}	Get edge by UUID or numeric alias.

```
curl $REST_BASE/zones
curl $REST_BASE/zones/100
curl $REST_BASE/nodes
curl $REST_BASE/nodes/1001
curl $REST_BASE/edges
curl $REST_BASE/edges/2001
```

Zone shape:

```
{
  "id": "00000000-0000-0000-0000-000000000100",
  "numeric_id": 100,
  "name": "dock_a",
  "kind": "zone",
  "parent_id": "00000000-0000-0000-0000-000000000001",
  "child_ids": [],
  "node_ids": [],
  "properties": {"traffic.policy": "exclusive"}
}
```

5.3 Route planning

Method	Path / Key	Request	Response
POST	/routes/plan	PlanRouteRequest	PlanRouteResponse

```
curl -X POST $REST_BASE/routes/plan -H 'content-type: application/json' -d
'{"start_node_id": "1001", "goal_node_id": "1003", "use_penalties": false}'
```

```
{
  "start_node_id": "1001",
  "goal_node_id": "1003",
  "use_penalties": false
}
```

Response fields:

found	Whether a route was found.
distance	Graph distance / cost.
plan	Route plan if found.
failure	Failure diagnostics if not found.

5.4 Robots

Method	Path	Meaning
GET	/robots	List registered robots.
POST	/robots	Register or replace robot state.
GET	/robots/{id}	Get robot state.
DELETE	/robots/{id}	Unregister robot.
POST	/robots/{id}/heartbeat	Update current node/edge and tick.

Method	Path	Meaning
POST	<code>/robots/{id}/route</code>	Assign a route plan.
POST	<code>/robots/{id}/schedule</code>	Schedule from an existing claim.

Register:

```
curl -X POST $REST_BASE/robots -H 'content-type: application/json' -d
'{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","up"
```

Heartbeat:

```
curl -X POST $REST_BASE/robots/1/heartbeat -H 'content-type: application/json' -d
'{"current_node_id":"1002","current_edge_id":null,"updated_at_tick":70}'
```

Schedule:

```
curl -X POST $REST_BASE/robots/1/schedule -H 'content-type: application/json' -d
'{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive"}'
```

5.5 Claims

Method	Path	Meaning
GET	<code>/claims</code>	List active claim requests.
POST	<code>/claims/evaluate</code>	Dry-run claim; does not store it.
POST	<code>/claims</code>	Evaluate and store only if granted.
GET	<code>/claims/{id}</code>	Get active claim request.
DELETE	<code>/claims/{id}</code>	Remove active claim request.

Submit claim:

```
curl -X POST $REST_BASE/claims -H 'content-type: application/json' -d
'{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}}]'
```

Claim body:

```
{
  "id": 10,
  "robot_id": 1,
  "mission_id": 9001,
  "access_mode": "Exclusive",
  "priority": 10,
  "requested_at_tick": 10,
  "window": {"start_tick": 10, "end_tick": 100},
  "targets": [{"kind": "Zone", "resource_id": "100"}]
}
```

5.6 Leases and snapshot

Method	Path	Meaning
GET	<code>/leases</code>	List leases.
POST	<code>/leases</code>	Add lease directly.
POST	<code>/leases/release</code>	Release lease by body.
DELETE	<code>/leases/{id}</code>	Release lease by path.
GET	<code>/fleet/snapshot</code>	Robots, active claim requests, and leases.

```
curl $REST_BASE/fleet/snapshot
```

6 XML protocol

Set the XML base URL:

```
XML_BASE=http://84.22.103.80:8080/ares/v1
```

XML uses the same `/ares/v1` prefix as REST. The server selects XML when:

- a `GET` request sends `Accept: application/xml`
- a request with a body sends `Content-Type: application/xml`

Body requests return XML automatically when their body was XML.

6.1 XML endpoint table

Method	Path	Request root
GET	<code>/health</code>	none
GET	<code>/fleet/snapshot</code>	none
POST	<code>/routes/plan</code>	<code><PlanRouteRequest></code>
GET	<code>/zones</code>	none
GET	<code>/zones/{id}</code>	none
GET	<code>/nodes</code>	none
GET	<code>/nodes/{id}</code>	none
GET	<code>/edges</code>	none
GET	<code>/edges/{id}</code>	none
GET	<code>/robots</code>	none
POST	<code>/robots</code>	<code><RobotState></code>
GET	<code>/robots/{id}</code>	none
DELETE	<code>/robots/{id}</code>	none
POST	<code>/robots/{id}/heartbeat</code>	<code><HeartbeatRequest></code>
POST	<code>/robots/{id}/route</code>	<code><AssignRouteRequest></code>
POST	<code>/robots/{id}/schedule</code>	<code><ScheduleRobotRouteRequest></code>
GET	<code>/claims</code>	none
POST	<code>/claims</code>	<code><ClaimRequestWire></code>
POST	<code>/claims/evaluate</code>	<code><ClaimRequestWire></code>
GET	<code>/claims/{id}</code>	none
DELETE	<code>/claims/{id}</code>	none
GET	<code>/leases</code>	none
POST	<code>/leases</code>	<code><Lease></code>
POST	<code>/leases/release</code>	<code><ReleaseLeaseRequest></code>
DELETE	<code>/leases/{id}</code>	none

6.2 XML calls

Health:

```
curl -H 'accept: application/xml' $XML_BASE/health
```

Zones:

```
curl -H 'accept: application/xml' $XML_BASE/zones
```

Get zone:

```
curl -H 'accept: application/xml' $XML_BASE/zones/100
```

Nodes:

```
curl -H 'accept: application/xml' $XML_BASE/nodes
```

Get node:

```
curl -H 'accept: application/xml' $XML_BASE/nodes/1001
```

Edges:

```
curl -H 'accept: application/xml' $XML_BASE/edges
```

Get edge:

```
curl -H 'accept: application/xml' $XML_BASE/edges/2001
```

Route:

```
curl -X POST $XML_BASE/routes/plan -H 'content-type: application/xml' -d
'<PlanRouteRequest><start_node_id>1001</start_node_id><goal_node_id>1003</
goal_node_id><use_penalties>false</use_penalties></PlanRouteRequest>'
```

Register robot:

```
curl -X POST $XML_BASE/robots -H 'content-type: application/xml' -d '<RobotState><robot_id>1</
robot_id><mission_id>9001</mission_id><current_node_id>00000000-0000-0000-0000-000000001001</
current_node_id><progress_state>Idle</progress_state><updated_at_tick>1</updated_at_tick></RobotState>'
```

List robots:

```
curl -H 'accept: application/xml' $XML_BASE/robots
```

Get robot:

```
curl -H 'accept: application/xml' $XML_BASE/robots/1
```

Unregister robot:

```
curl -X DELETE -H 'accept: application/xml' $XML_BASE/robots/1
```

Heartbeat:

```
curl -X POST $XML_BASE/robots/1/heartbeat -H 'content-type: application/xml' -d
'<HeartbeatRequest><current_node_id>1002</current_node_id><updated_at_tick>70</updated_at_tick></
HeartbeatRequest>'
```

Assign route:

```
curl -X POST $XML_BASE/robots/1/route -H 'content-type: application/xml' -d
'<AssignRouteRequest><route_plan><start_node_id>00000000-0000-0000-0000-000000001001</
start_node_id><goal_node_id>00000000-0000-0000-0000-000000001003</goal_node_id><total_cost>0</
total_cost></route_plan><horizon>100</horizon><updated_at_tick>10</updated_at_tick></AssignRouteRequest>'
```

Schedule:

```
curl -X POST $XML_BASE/robots/1/schedule -H 'content-type: application/xml' -d
'<ScheduleRobotRouteRequest><claim_id>10</claim_id><start_tick>20</start_tick><ticks_per_cost_unit>1.0</ticks_per_cost_unit><access_mode>Exclusive</access_mode></ScheduleRobotRouteRequest>'
```

Claim evaluate:

```
curl -X POST $XML_BASE/claims/evaluate -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

Submit claim:

```
curl -X POST $XML_BASE/claims -H 'content-type: application/xml' -d '<ClaimRequestWire><id>10</id><robot_id>1</robot_id><mission_id>9001</mission_id><access_mode>Exclusive</access_mode><priority>10</priority><requested_at_tick>10</requested_at_tick><window><start_tick>10</start_tick><end_tick>100</end_tick></window><targets><kind>Zone</kind><resource_id>100</resource_id></targets></ClaimRequestWire>'
```

List claims:

```
curl -H 'accept: application/xml' $XML_BASE/claims
```

Get claim:

```
curl -H 'accept: application/xml' $XML_BASE/claims/10
```

Remove claim:

```
curl -X DELETE -H 'accept: application/xml' $XML_BASE/claims/10
```

List leases:

```
curl -H 'accept: application/xml' $XML_BASE/leases
```

Add lease:

```
curl -X POST $XML_BASE/leases -H 'content-type: application/xml' -d '<Lease><id>501</id><claim_id>10</claim_id><robot_id>1</robot_id><access_mode>Exclusive</access_mode><targets><kind>Zone</kind><resource_id>00000000-0000-0000-0000-000000000100</resource_id></targets><granted_at_tick>12</granted_at_tick><expires_at_tick>200</expires_at_tick><disposition>Active</disposition><active>true</active></Lease>'
```

Lease release:

```
curl -X POST $XML_BASE/leases/release -H 'content-type: application/xml' -d
'<ReleaseLeaseRequest><lease_id>501</lease_id><released_at_tick>50</released_at_tick></ReleaseLeaseRequest>'
```

Release lease by path:

```
curl -X DELETE -H 'accept: application/xml' $XML_BASE/leases/501
```

7 Zenoh and ROS2DDS

The ARES server stays pure Zenoh. It does not link ROS2. ROS2 machines call it through `zenoh-bridge-ros2dds`, which translates a ROS2 service request into a Zenoh query.

7.1 ROS2 service catalog

This is the complete ROS2DDS service surface currently exposed by ARES.

ROS2 service	ROS2 type	Zenoh key	Meaning
/ares/v1/health	ares_interfaces/srv/Json	ares/v1/health	Health/version as JSON.
/ares/v1/fleet/snapshot	ares_interfaces/srv/Json	ares/v1/fleet/snapshot	Robots, claims, and leases.
/ares/v1/zones/list	ares_interfaces/srv/Json	ares/v1/zones/list	List zones.
/ares/v1/zones/get	ares_interfaces/srv/Json	ares/v1/zones/get	Get zone by id.
/ares/v1/nodes/list	ares_interfaces/srv/Json	ares/v1/nodes/list	List nodes.
/ares/v1/nodes/get	ares_interfaces/srv/Json	ares/v1/nodes/get	Get node by id.
/ares/v1/edges/list	ares_interfaces/srv/Json	ares/v1/edges/list	List edges.
/ares/v1/edges/get	ares_interfaces/srv/Json	ares/v1/edges/get	Get edge by id.
/ares/v1/routes/plan	ares_interfaces/srv/Json	ares/v1/routes/plan	Plan route from JSON request.
/ares/v1/robots/register	ares_interfaces/srv/Json	ares/v1/robots/register	Register robot from JSON request.
/ares/v1/robots/list	ares_interfaces/srv/Json	ares/v1/robots/list	List robots.
/ares/v1/robots/get	ares_interfaces/srv/Json	ares/v1/robots/get	Get robot.
/ares/v1/robots/unregister	ares_interfaces/srv/Json	ares/v1/robots/unregister	Unregister robot.
/ares/v1/robots/heartbeat	ares_interfaces/srv/Json	ares/v1/robots/heartbeat	Update robot progress.
/ares/v1/robots/assign_route	ares_interfaces/srv/Json	ares/v1/robots/assign_route	Assign route plan.
/ares/v1/robots/schedule	ares_interfaces/srv/Json	ares/v1/robots/schedule	Schedule robot from claim.
/ares/v1/claims/list	ares_interfaces/srv/Json	ares/v1/claims/list	List active claims.
/ares/v1/claims/get	ares_interfaces/srv/Json	ares/v1/claims/get	Get active claim.
/ares/v1/claims/remove	ares_interfaces/srv/Json	ares/v1/claims/remove	Remove active claim.
/ares/v1/claims/evaluate	ares_interfaces/srv/Json	ares/v1/claims/evaluate	Dry-run claim.
/ares/v1/claims/request	ares_interfaces/srv/Json	ares/v1/claims/request	Submit claim if granted.
/ares/v1/leases/list	ares_interfaces/srv/Json	ares/v1/leases/list	List leases.
/ares/v1/leases/add	ares_interfaces/srv/Json	ares/v1/leases/add	Add lease.
/ares/v1/leases/release	ares_interfaces/srv/Json	ares/v1/leases/release	Release lease.

The /ares/v1/... services are the real ARES ROS2DDS service surface.

7.2 ares_interfaces/srv/Json

The full ARES ROS2 service surface uses one generic JSON service type:

```
string request
---
bool success
string response
```

The `.srv` file is included in this repository at:

```
ros/ares_interfaces/srv/Json.srv
```

Rules:

<code>request</code>	JSON request object encoded as a string. Use <code>{}</code> for no-input services.
<code>success</code>	<code>true</code> when the call succeeded; <code>false</code> when ARES returned an error.
<code>response</code>	On success: JSON response encoded as a string. On error: error text.

Example call:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

7.3 Full ROS2 JSON services

Health:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

Fleet snapshot:

```
ros2 service call /ares/v1/fleet/snapshot ares_interfaces/srv/Json "{request: '{}'}"
```

List zones:

```
ros2 service call /ares/v1/zones/list ares_interfaces/srv/Json "{request: '{}'}"
```

Get zone:

```
ros2 service call /ares/v1/zones/get ares_interfaces/srv/Json "{request: '{\"id\":\"100\"}'}"
```

List nodes:

```
ros2 service call /ares/v1/nodes/list ares_interfaces/srv/Json "{request: '{}'}"
```

Get node:

```
ros2 service call /ares/v1/nodes/get ares_interfaces/srv/Json "{request: '{\"id\":\"1001\"}'}"
```

List edges:

```
ros2 service call /ares/v1/edges/list ares_interfaces/srv/Json "{request: '{}'}"
```

Get edge:

```
ros2 service call /ares/v1/edges/get ares_interfaces/srv/Json "{request: '{\"id\":\"2001\"}'}"
```

Route planning:

```
ros2 service call /ares/v1/routes/plan ares_interfaces/srv/Json "{request: '{\"start_node_id\":\"1001\", \"goal_node_id\":\"1003\", \"use_penalties\":false}'}"
```

Register robot:

```
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json "{request: '{\"robot_id\":1,
\"mission_id\":9001,\"current_node_id\": \"00000000-0000-0000-0000-000000001001\", \"progress_state\":
\"Idle\", \"updated_at_tick\":1}'}"
```

List robots:

```
ros2 service call /ares/v1/robots/list ares_interfaces/srv/Json "{request: '{}'}"
```

Get robot:

```
ros2 service call /ares/v1/robots/get ares_interfaces/srv/Json "{request: '{\"robot_id\":1}'}"
```

Unregister robot:

```
ros2 service call /ares/v1/robots/unregister ares_interfaces/srv/Json "{request: '{\"robot_id\":1}'}"
```

Heartbeat:

```
ros2 service call /ares/v1/robots/heartbeat ares_interfaces/srv/Json "{request: '{\"robot_id\":1,
\"heartbeat\":{\"current_node_id\": \"1002\", \"current_edge_id\":null, \"updated_at_tick\":70}'}"
```

Assign route:

```
ros2 service call /ares/v1/robots/assign_route ares_interfaces/srv/Json "{request: '{\"robot_id\":1,
\"assignment\":{\"route_plan\":{\"start_node_id\": \"00000000-0000-0000-0000-000000001001\",
\"goal_node_id\": \"00000000-0000-0000-0000-000000001003\", \"steps\": [], \"traversed_node_ids\": [],
\"traversed_edge_ids\": [], \"traversed_zone_ids\": [], \"traversed_node_zone_ids\": [],
\"traversed_edge_zone_ids\": [], \"total_cost\":0.0}, \"horizon\":100, \"updated_at_tick\":10}'}"
```

Schedule:

```
ros2 service call /ares/v1/robots/schedule ares_interfaces/srv/Json "{request: '{\"robot_id\":1,
\"schedule\":{\"claim_id\":10, \"start_tick\":20, \"ticks_per_cost_unit\":1.0, \"access_mode\":
\"Exclusive\"}'}"
```

List claims:

```
ros2 service call /ares/v1/claims/list ares_interfaces/srv/Json "{request: '{}'}"
```

Get claim:

```
ros2 service call /ares/v1/claims/get ares_interfaces/srv/Json "{request: '{\"claim_id\":10}'}"
```

Remove claim:

```
ros2 service call /ares/v1/claims/remove ares_interfaces/srv/Json "{request: '{\"claim_id\":10}'}"
```

Evaluate claim:

```
ros2 service call /ares/v1/claims/evaluate ares_interfaces/srv/Json "{request: '{\"id\":10,
\"robot_id\":1, \"mission_id\":9001, \"access_mode\": \"Exclusive\", \"priority\":10,
\"requested_at_tick\":10, \"window\":{\"start_tick\":10, \"end_tick\":100}, \"targets\":[{\"kind\": \"Zone\",
\"resource_id\": \"100\"}]}'"
```

Submit claim:

```
ros2 service call /ares/v1/claims/request ares_interfaces/srv/Json "{request: '{\"id\":10, \"robot_id\":1,
\"mission_id\":9001, \"access_mode\": \"Exclusive\", \"priority\":10, \"requested_at_tick\":10, \"window\":
{\"start_tick\":10, \"end_tick\":100}, \"targets\":[{\"kind\": \"Zone\", \"resource_id\": \"100\"}]}'"
```

List leases:

```
ros2 service call /ares/v1/leases/list ares_interfaces/srv/Json "{request: '{}'}"
```

Add lease:

```
ros2 service call /ares/v1/leases/add ares_interfaces/srv/Json "{request: '{\"id\":501,\"claim_id\":10,
\"robot_id\":1,\"access_mode\":\"Exclusive\", \"targets\": [{\"kind\":\"Zone\", \"resource_id\":
\"00000000-0000-0000-0000-000000000100\"}], \"granted_at_tick\":12, \"expires_at_tick\":200,
\"disposition\":\"Active\", \"active\":true}'}"
```

Release lease:

```
ros2 service call /ares/v1/leases/release ares_interfaces/srv/Json "{request: '{\"lease_id\":501,
\"released_at_tick\":50}'}"
```

7.4 ROS2 connection story

On the ARES machine, the server listens for Zenoh peers on `tcp/0.0.0.0:7447`. On the ROS2 machine, connect the bridge to the ARES machine:

```
zenoh-bridge-ros2dds -e tcp/TIMENAV_IP:7447
```

Use `tcp/TIMENAV_IP:7447`, not `tcp://TIMENAV_IP:7447`.

In another ROS2 terminal, source ROS and call the service:

```
ros2 service call /ares/v1/health ares_interfaces/srv/Json "{request: '{}'}"
```

Expected CLI shape:

```
requester: making request: ares_interfaces.srv.Json_Request(request='{}')

response:
ares_interfaces.srv.Json_Response(
  success=true,
  response='{\"status\":\"ok\", \"version\":\"0.0.2\"}'
)
```

If the bridge logs a route like this, the ROS2-to-Zenoh path is working:

```
Route Service Client (ROS:/ares/v1/health <-> Zenoh:ares/v1/health) created
```

`ros2 service list` may not show the ARES services. That is expected in the pure-Zenoh setup. The call works because the bridge creates the route dynamically when the request is made.

7.5 ROS2 troubleshooting

Symptom	Fix / explanation
Unicast not supported for tcp: protocol	The endpoint used <code>tcp://...</code> . Use <code>tcp/...</code> instead.
<code>ros2 service list</code> does not show the service	Expected for this pure-Zenoh setup. Call the <code>/ares/v1/...</code> service directly.
waiting for service to become available...	Keep the bridge running, then call directly. The bridge creates the route when it sees the client.
received invalid request ... less than 20 bytes	Use CycloneDDS on the ROS2 side: <code>export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp</code> .
No response	Check firewall/NAT and confirm the bridge can reach <code>tcp/TIMENAV_IP:7447</code> .

7.6 Internal ROS2DDS response encoding

ARES replies with a CDR-encoded `ares_interfaces/srv/Json_Response`:

```
00 01 00 00      CDR little-endian header
01              bool success
```

```

00 00 00          padding to 4 bytes
<u32 len including NUL>  string length
<UTF-8 JSON bytes>
00                trailing NUL
<padding to 4 bytes>

```

8 ARES native Zenoh services

These are the ARES native Zenoh services from the `robo` adapter. They are JSON queryables under the `ares/v1` prefix. Request payloads are JSON strings where the table says a payload is required. Replies are JSON. Errors are Zenoh error replies with `{"message": "..."}.`

8.1 Complete Zenoh service catalog

Key	Payload	Response
<code>ares/v1/health</code>	none	<code>Health</code>
<code>ares/v1/fleet/snapshot</code>	none	<code>FleetSnapshot</code>
<code>ares/v1/zones/list</code>	none	<code>ZoneView[]</code>
<code>ares/v1/zones/get</code>	<code>{"id": "100"}</code>	<code>ZoneView</code>
<code>ares/v1/nodes/list</code>	none	<code>NodeView[]</code>
<code>ares/v1/nodes/get</code>	<code>{"id": "1001"}</code>	<code>NodeView</code>
<code>ares/v1/edges/list</code>	none	<code>EdgeView[]</code>
<code>ares/v1/edges/get</code>	<code>{"id": "2001"}</code>	<code>EdgeView</code>
<code>ares/v1/routes/plan</code>	<code>PlanRouteRequest</code> JSON	<code>PlanRouteResponse</code>
<code>ares/v1/robots/register</code>	<code>RobotState</code> JSON	<code>RobotState</code>
<code>ares/v1/robots/list</code>	none	<code>RobotState[]</code>
<code>ares/v1/robots/get</code>	<code>{"robot_id": 1}</code>	<code>RobotState</code>
<code>ares/v1/robots/unregister</code>	<code>{"robot_id": 1}</code>	<code>bool</code>
<code>ares/v1/robots/heartbeat</code>	<code>{"robot_id": 1, "heartbeat": ...}</code>	<code>RobotState</code>
<code>ares/v1/robots/assign_route</code>	<code>{"robot_id": 1, "assignment": ...}</code>	<code>RobotState</code>
<code>ares/v1/robots/schedule</code>	<code>{"robot_id": 1, "schedule": ...}</code>	<code>ScheduleDecision</code>
<code>ares/v1/claims/list</code>	none	<code>ClaimRequest[]</code>
<code>ares/v1/claims/get</code>	<code>{"claim_id": 10}</code>	<code>ClaimRequest</code>
<code>ares/v1/claims/remove</code>	<code>{"claim_id": 10}</code>	<code>bool</code>
<code>ares/v1/claims/evaluate</code>	<code>ClaimRequestWire</code> JSON	<code>ClaimEvaluation</code>
<code>ares/v1/claims/request</code>	<code>ClaimRequestWire</code> JSON	<code>ClaimEvaluation</code>
<code>ares/v1/leases/list</code>	none	<code>Lease[]</code>
<code>ares/v1/leases/add</code>	<code>Lease</code> JSON	<code>Lease</code>
<code>ares/v1/leases/release</code>	<code>ReleaseLeaseRequest</code> JSON	<code>bool</code>

8.1.1 `ares/v1/health`

Checks that the ARES service is alive.

Payload	none
Response	<code>Health: {"status": "ok", "version": "0.0.2"}</code>

8.1.2 ares/v1/fleet/snapshot

Returns the whole live coordination state: robots, active claim requests, and leases.

Payload	none
Response	FleetSnapshot: {"robots":[],"requests":[],"leases":[]}

8.1.3 ares/v1/zones/list

Lists zones.

Payload	none
Response	ZoneView[] JSON

8.1.4 ares/v1/zones/get

Gets one zone by UUID or numeric alias.

Payload	{"id":"100"}
Response	ZoneView JSON

8.1.5 ares/v1/nodes/list

Lists nodes.

Payload	none
Response	NodeView[] JSON

8.1.6 ares/v1/nodes/get

Gets one node by UUID or numeric alias.

Payload	{"id":"1001"}
Response	NodeView JSON

8.1.7 ares/v1/edges/list

Lists edges.

Payload	none
Response	EdgeView[] JSON

8.1.8 ares/v1/edges/get

Gets one edge by UUID or numeric alias.

Payload	{"id":"2001"}
Response	EdgeView JSON

8.1.9 ares/v1/routes/plan

Plans a route between two graph nodes.

Payload	PlanRouteRequest JSON
Response	PlanRouteResponse JSON

```
{"start_node_id":"1001","goal_node_id":"1003","use_penalties":false}
```

8.1.10 ares/v1/robots/register

Registers or replaces a robot state.

Payload	RobotState JSON
Response	RobotState JSON

```
{"robot_id":1,"mission_id":9001,"current_node_id":"00000000-0000-0000-0000-000000001001","progress_state":"Idle","update_at_tick":70}
```

8.1.11 ares/v1/robots/list

Lists all registered robots.

Payload	none
Response	RobotState[] JSON

8.1.12 ares/v1/robots/get

Gets one robot.

Payload	{"robot_id":1}
Response	RobotState JSON

8.1.13 ares/v1/robots/unregister

Unregisters one robot.

Payload	{"robot_id":1}
Response	bool

8.1.14 ares/v1/robots/heartbeat

Updates the robot's current node/edge and timestamp.

Payload	RobotHeartbeatEnvelope JSON
Response	RobotState JSON

```
{"robot_id":1,"heartbeat":{"current_node_id":"1002","current_edge_id":null,"updated_at_tick":70}}
```

8.1.15 ares/v1/robots/assign_route

Assigns an already computed route plan to a robot.

Payload	RobotAssignRouteEnvelope JSON
Response	RobotState JSON

```
{
  "robot_id": 1,
  "assignment": {
    "route_plan": {
      "start_node_id": "00000000-0000-0000-0000-000000001001",
      "goal_node_id": "00000000-0000-0000-0000-000000001003",
      "steps": [],
      "traversed_node_ids": [],
      "traversed_edge_ids": [],
      "traversed_zone_ids": [],
      "traversed_node_zone_ids": [],
      "traversed_edge_zone_ids": [],
      "total_cost": 0.0
    }
  }
}
```

```

    },
    "horizon": 100,
    "updated_at_tick": 10
  }
}

```

8.1.16 `ares/v1/robots/schedule`

Schedules a robot from an existing claim.

Payload	RobotScheduleEnvelope JSON
Response	ScheduleDecision JSON

```

{"robot_id":1,"schedule":
{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive"}}

```

8.1.17 `ares/v1/claims/list`

Lists active claim requests.

Payload	none
Response	ClaimRequest[] JSON

8.1.18 `ares/v1/claims/get`

Gets one active claim request.

Payload	{"claim_id":10}
Response	ClaimRequest JSON

8.1.19 `ares/v1/claims/remove`

Removes one active claim request.

Payload	{"claim_id":10}
Response	bool

8.1.20 `ares/v1/claims/evaluate`

Dry-runs a claim without storing it.

Payload	ClaimRequestWire JSON
Response	ClaimEvaluation JSON

```

{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}

```

8.1.21 `ares/v1/claims/request`

Evaluates a claim and stores it if the decision is `Grant`.

Payload	ClaimRequestWire JSON
Response	ClaimEvaluation JSON

```

{"id":10,"robot_id":1,"mission_id":9001,"access_mode":"Exclusive","priority":10,"requested_at_tick":10,"window":
{"start_tick":10,"end_tick":100},"targets":[{"kind":"Zone","resource_id":"100"}]}

```

8.1.22 ares/v1/leases/list

Lists leases.

Payload	none
Response	Lease[] JSON

8.1.23 ares/v1/leases/add

Adds a lease directly.

Payload	Lease JSON
Response	Lease JSON

```
{ "id": 501, "claim_id": 10, "robot_id": 1, "access_mode": "Exclusive", "targets":
  [{ "kind": "Zone", "resource_id": "00000000-0000-0000-0000-000000000100" }], "granted_at_tick": 12, "expires_at_tick": 200, "dis
```

8.1.24 ares/v1/leases/release

Releases a lease.

Payload	ReleaseLeaseRequest JSON
Response	bool

```
{ "lease_id": 501, "released_at_tick": 50 }
```

8.2 Native Zenoh operating sequence

The native Zenoh story mirrors REST:

Step	Zenoh key	Payload
1	ares/v1/health	none
2	ares/v1/robots/register	RobotState
3	ares/v1/routes/plan	PlanRouteRequest
4	ares/v1/claims/evaluate	ClaimRequestWire dry-run
5	ares/v1/claims/request	ClaimRequestWire store-if-granted
6	ares/v1/robots/heartbeat	RobotHeartbeatEnvelope
7	ares/v1/fleet/snapshot	none
8	ares/v1/leases/release	ReleaseLeaseRequest, if leases are used